# CAAS: ChiangMai Air Quality Alert System

An End-to-End MLOps Pipeline for PM2.5 Forecasting

Supanut Kompayak (st126055)

Shuvam Shrestha (st125975)

AT82.9002 Data Engineering and MLOps

Asian Institute of Technology

April 9, 2026

# Contents


**1 Introduction** **1**
1.1 Problem Statement . . . 1
1.2 Objectives . . . 1
1.3 Contributions . . . 1

**2 Data Engineering** **3**
2.1 Data Sources . . . 3
2.2 Data Ingestion Pipeline . . . 3
2.3 Feature Engineering . . . 4
2.4 Data Storage and Versioning . . . 4
2.5 Pipeline Orchestration . . . 5

**3 Machine Learning Model Development** **6**
3.1 Dataset and Splits . . . 6
3.2 Baseline . . . 6
3.3 Model Selection . . . 6
3.4 Results . . . 7
3.4.1 XGBoost (Champion) . . . 7
3.4.2 LSTM (Comparison) . . . 7
3.4.3 Actual vs. Predicted . . . 8
3.5 Ablation Study: FIRMS Fire Data . . . 8
3.6 Alert Classifier Analysis (Scenario C) . . . 9
3.7 Model Interpretation (SHAP) . . . 9

**4 MLOps Implementation** **12**
4.1 System Architecture . . . 12
4.2 Experiment Tracking and Model Registry . . . 12
4.3 Model Serving . . . 13
4.4 CI/CD Pipeline . . . 14
4.5 Monitoring and Drift Detection . . . 15
4.6 Infrastructure as Code . . . 16

**5 Performance Analysis** **17**
5.1 Model Performance Summary . . . 17
5.2 Inference Latency . . . 17
5.3 Scalability Analysis . . . 18
5.4 Cost Analysis . . . 18

**6 Conclusion** **19**
6.1 Summary of Findings . . . 19
6.2 Limitations . . . 19
6.3 Future Work . . . 20

**References** **21**

**A Reproduction Guide** **24**
A.1 Prerequisites . . . 24
A.2 Step 1 — Clone and Environment Setup . . . 24
A.3 Step 2 — Configure Environment Variables . . . 24
A.4 Step 3 — Provision AWS Infrastructure with Terraform . . . 25
A.5 Step 4 — Historical Data Backfill . . . 25
A.6 Step 5 — Train Models and Register with MLflow . . . 25
A.7 Step 6 — Run Analysis Scripts (Optional) . . . 25
A.8 Step 7 — Run Inference and Upload to S3 . . . 26
A.9 Step 8 — Build and Deploy Docker Container . . . 26
A.10 Step 9 — Enable CI/CD Automation . . . 26
A.11 Step 10 — Launch the Streamlit Dashboard . . . 27
A.12 Step 11 — Run the Test Suite . . . 27

**B Full Metric Tables** **28**
B.1 XGBoost — Validation and Test Metrics . . . 28
B.2 LSTM — Validation and Test Metrics . . . 28
B.3 Alert Classification Metrics (XGBoost, 50 µg/m$^3$ threshold) . . . 28
B.4 SHAP Mean Absolute Values — Top 5 Features per Horizon . . . 29

**Abstract**

Chiang Mai, Thailand experiences severe annual PM2.5 pollution events driven by agricultural burning, with concentrations frequently exceeding the WHO 24-hour guideline of 15 µg/m$^3$ by an order of magnitude. This report presents CAAS (ChiangMai Air Quality Alert System), a production-grade end-to-end MLOps pipeline for multi-horizon PM2.5 forecasting and binary hazard alerting (PM2.5 $>$ 50 µg/m$^3$). CAAS integrates three data sources — PCD Thailand PM2.5 measurements, Open-Meteo ERA5 weather reanalysis, and NASA FIRMS satellite fire hotspot data — into a 45-feature daily time-series matrix spanning 2011–2025. An XGBoost champion model outperforms an LSTM comparison model at all forecast horizons, achieving MAE of 5.31, 6.91, and 8.34 µg/m$^3$ at t+1, t+3, and t+7 days respectively ($R^2$ = 0.824 / 0.736 / 0.612, AUROC = 0.991 / 0.976 / 0.959 on the 2024–2025 test set). SHAP analysis reveals a physically interpretable shift in dominant features: recent PM2.5 persistence (t+1), 14-day fire accumulation (t+3), and seasonal calendar position (t+7). The MLOps stack — MLflow experiment tracking, FastAPI inference, Evidently drift monitoring, and Terraform-provisioned AWS infrastructure orchestrated by GitHub Actions — constitutes a deployable, self-retraining system operating at approximately $0.55/day.

# Chapter 1

# Introduction

## 1.1 Problem Statement

Chiang Mai, Thailand, experiences severe seasonal air pollution events driven by agricultural burning and forest fires between February and April each year. PM2.5 concentrations routinely exceed 150 µg/m$^3$ during haze season — more than six times the WHO guideline of 15 µg/m$^3$ and three times Thailand's national standard of 50 µg/m$^3$. The health impact is substantial: elevated PM2.5 is associated with respiratory disease, cardiovascular events, and premature mortality, placing particular burden on children, the elderly, and outdoor workers.

Despite the severity of the problem, no publicly accessible automated forecasting service provides Chiang Mai residents with reliable multi-day PM2.5 predictions. Current government monitoring (air4thai, PCD Thailand) reports measured values but does not forecast future concentrations or issue advance hazard warnings, leaving citizens unable to plan protective behaviour.

## 1.2 Objectives

This project — the **ChiangMai Air Quality Alert System (CAAS)** — addresses this gap by building a complete, end-to-end MLOps pipeline that:

1. **Forecasts PM2.5** at three horizons: next day (t+1), three days ahead (t+3), and seven days ahead (t+7), using 15 years of historical data (2011–2025).
2. **Issues binary hazard alerts** when PM2.5 is forecast to exceed 50 µg/m$^3$, enabling early protective action.
3. **Integrates NASA FIRMS fire hotspot data** alongside meteorological variables to capture the dominant driver of seasonal PM2.5 spikes.
4. **Deploys as a production-grade MLOps pipeline** with automated data ingestion, drift monitoring, champion/challenger model governance, and CI/CD-triggered retraining.

## 1.3 Contributions

The key contributions of this project are:

- A 45-feature dataset spanning 5,257 days (2011–2025), merging three heterogeneous sources: PCD PM2.5 measurements, Open-Meteo weather reanalysis, and NASA FIRMS fire hotspot counts.

- An XGBoost champion model achieving MAE of 5.31, 6.91, and 8.34 µg/m$^3$ at t+1, t+3, and t+7 respectively, with AUROC $\geq$ 0.96 across all horizons.
- A SHAP-based interpretability analysis demonstrating that the dominant predictive signal shifts from recent PM2.5 lags (short horizons) to fire hotspot accumulation and seasonal position (longer horizons).
- An ablation study quantifying the contribution of FIRMS fire data: removing six fire-derived features degrades MAE by 4.3–6.8% across horizons.
- A complete MLOps stack: FastAPI inference service, Streamlit monitoring dashboard, Evidently drift monitoring with automated GitHub Actions retraining triggers, and Terraform-provisioned AWS infrastructure.

# Chapter 2

# Data Engineering

## 2.1 Data Sources

CAAS ingests data from three independent sources:

**1. PM2.5 Measurements — PCD Thailand.** Daily PM2.5 concentration data for Chiang Mai stations 35T and 36T was obtained from the Pollution Control Department's air4thai portal, covering 2011–2025. Historical data was provided as Excel files; live daily updates are fetched via the air4thai JSON API (`fetch_pm25_live.py`).

**2. Meteorological Data — Open-Meteo.** Historical and forecast weather variables were obtained from the Open-Meteo API, an open-access service providing ERA5-quality reanalysis data at daily resolution. Variables collected include temperature (max/min/mean), humidity (max/min), precipitation, wind speed, wind direction, surface pressure, cloud cover, and evapotranspiration (`fetch_weather.py`).

**3. Fire Hotspot Data — NASA FIRMS.** Daily fire hotspot counts within 50 km and 100 km radii of Chiang Mai were obtained from the NASA FIRMS API (VIIRS/SNPP satellite product). For each hotspot radius, mean Fire Radiative Power (FRP) was also extracted as a proxy for fire intensity. Historical data was batch-fetched by year (`run_firms_batched.py`); daily updates use `fetch_firms.py`.

## 2.2 Data Ingestion Pipeline

Figure 2.1 illustrates the daily ingestion workflow. For historical backfill, `run_pipeline.sh` orchestrates the full sequence (Steps 1–4) in order. For production operation, a GitHub Actions cron job (`daily_pipeline.yml`) runs every day at 06:00 Bangkok time (23:00 UTC).

The ingestion pipeline:

1. Fetches yesterday's PM2.5, weather, and FIRMS data in parallel.
2. Appends new rows to the consolidated CSVs, deduplicating by date.
3. Rebuilds the feature matrix (`build_features.py`).
4. Runs inference with the current champion model.
5. Uploads all results to Amazon S3.
6. Runs the Evidently drift check and triggers retraining if thresholds are exceeded.

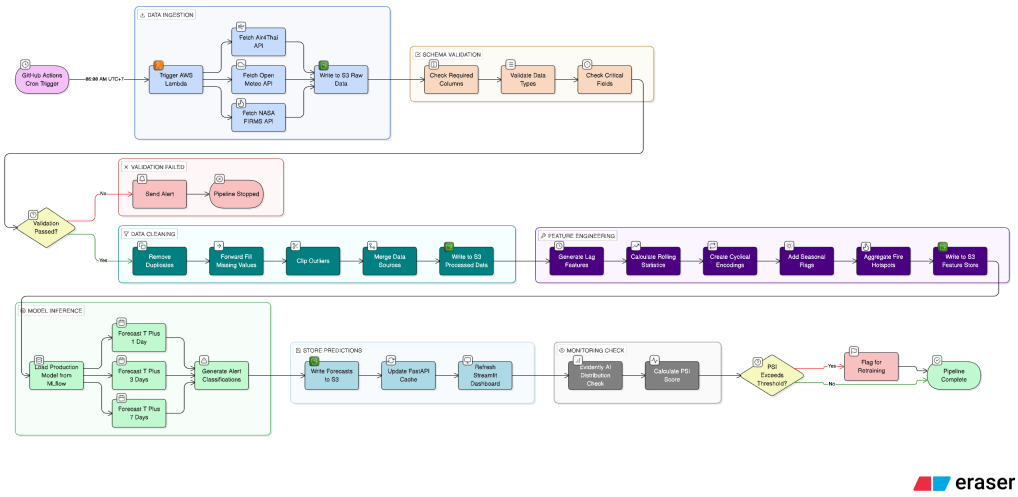


Figure 2.1: CAAS daily data pipeline — from ingestion to drift monitoring.

## 2.3 Feature Engineering

The feature matrix (`build_features.py`) produces 45 features from the three raw sources, grouped as follows:

Table 2.1: CAAS Feature Groups

| Group | Features |
|---|---|
| PM2.5 lags | pm25_lag1/3/7/14/30 |
| Rolling statistics | roll3/7/14/30_mean, roll7/14_std |
| Temporal encodings | month, week, day_of_year, sin/cos encodings, is_haze_season |
| Weather | temp (max/min/mean), humidity (max/min), precipitation, wind speed, wind directio |
| Fire (FIRMS) | hotspot_50km, hotspot_100km, mean_frp_50km, hotspot_7d_roll, hotspot_14d_ro |
| Alert lags | alert_lag1/3/7 |
| Cross features | lag1×temp, lag1×wind, roll7×fire, precip_flag, humidity×wind |
| **Total** | |

Key engineering decisions:

- **No data leakage:** Lag and rolling features use only past values. Target columns (pm25_t1, pm25_t3, pm25_t7) are forward-shifted.
- **Fire accumulation features:** 7-day and 14-day rolling hotspot counts capture the cumulative smoke load better than a single-day count.
- **Cyclic temporal encodings:** Month and day-of-year are encoded as sine/cosine pairs to preserve calendar continuity (December-January boundary).
- **Cross features:** lag1×temp and roll7×fire capture interactive effects between PM2.5 persistence and environmental conditions.

## 2.4 Data Storage and Versioning

All data artefacts are stored in Amazon S3 with a structured prefix layout:

```
s3://caas-mlops-bucket/
    data/raw/         ← original Excel and CSV files
    data/processed/   ← pm25_consolidated.csv, weather, FIRMS
    data/features/    ← features.csv (daily snapshot)
    models/           ← xgboost_t1/3/7.json (champion models)
    results/          ← forecasts, drift reports, summaries
    mlflow/           ← MLflow artefacts
```

S3 bucket versioning is enabled, preserving every version of model files and allowing rollback to any previous champion.

## 2.5 Pipeline Orchestration

Pipeline scheduling is handled by GitHub Actions:

- **daily_pipeline.yml** — cron 06:00 Bangkok time, runs ingest + inference + upload + drift check.
- **retrain.yml** — triggered annually (December 1) or on-demand via `repository_dispatch` from the drift monitor.
- **test.yml** — runs pytest suite on every push to main.

This architecture was chosen over Apache Airflow for simplicity within the capstone scope. Airflow would be appropriate for multi-team production use requiring finer DAG-level observability and retries.

# Chapter 3

# Machine Learning Model Development

## 3.1 Dataset and Splits

The feature matrix contains 5,257 daily records spanning May 2011 to December 2025. A strict chronological split was applied to prevent data leakage:

Table 3.1: Train / Validation / Test Split

| **Split** | **Period** | **Rows** |
|---|---|---|
| Train | 2011-05-18 → 2022-12-31 | 4,160 |
| Validation | 2023-01-01 → 2023-12-31 | 365 |
| Test | 2024-01-01 → 2025-12-31 | 731 |

All preprocessing (median imputation, scaling) was fitted exclusively on the training set and applied to validation and test sets without look-ahead.

## 3.2 Baseline

A persistence baseline was established using yesterday's PM2.5 value as the forecast for all horizons. This represents the simplest possible model and provides a floor that any learned model must surpass.

Table 3.2: Persistence Baseline — Test MAE

| **Horizon** | **MAE (µg/m³)** |
|---|---|
| t+1 | 5.30 |
| t+3 | 8.97 |
| t+7 | 10.93 |

## 3.3 Model Selection

Two model families were evaluated:

**XGBoost** was selected as the primary candidate due to its strong performance on tabular time-series data, native handling of missing values, interpretability via SHAP, and fast inference. Hyperparameters were tuned per horizon: t+1 and t+3 share base parameters (500 estimators, max_depth=6, lr=0.05), while t+7 uses a regularised configuration (1000 estimators, max_depth=4, lr=0.03) to reduce overfitting on the harder long-horizon task.

**LSTM** was trained as a comparison model to assess whether the temporal sequence structure adds value over tabular features. Architecture: three LSTM layers [128, 64, 32] with BatchNormalization and Dropout(0.2) between layers, 60-day lookback window, Huber loss (robust to PM2.5 spikes).

## 3.4 Results

### 3.4.1 XGBoost (Champion)

Table 3.3: XGBoost Test Set Performance

| Horizon | MAE | RMSE | $R^2$ | AUROC |
|---|---|---|---|---|
| t+1 | 5.31 | 9.48 | 0.824 | 0.991 |
| t+3 | 6.91 | 11.62 | 0.736 | 0.976 |
| t+7 | 8.34 | 14.13 | 0.612 | 0.959 |

XGBoost beats the persistence baseline at every horizon: MAE improvement is +0% at t+1 (near-parity due to the strong lag-1 signal), +23% at t+3, and +24% at t+7.

### 3.4.2 LSTM (Comparison)

Table 3.4: LSTM Test Set Performance

| Horizon | MAE | RMSE | $R^2$ |
|---|---|---|---|
| t+1 | 6.67 | 11.64 | 0.753 |
| t+3 | 8.87 | 15.89 | 0.540 |
| t+7 | 8.06 | 14.49 | 0.608 |

XGBoost outperforms LSTM at t+1 (MAE 5.31 vs. 6.67) and t+3 (6.91 vs. 8.87), while performance at t+7 is comparable (8.34 vs. 8.06 MAE). The LSTM's weaker t+3 result ($R^2$=0.54) reflects the difficulty of the mid-range horizon, where the recency of lag features decays but seasonal signals are not yet dominant. XGBoost is selected as the champion model.

### 3.4.3 Actual vs. Predicted

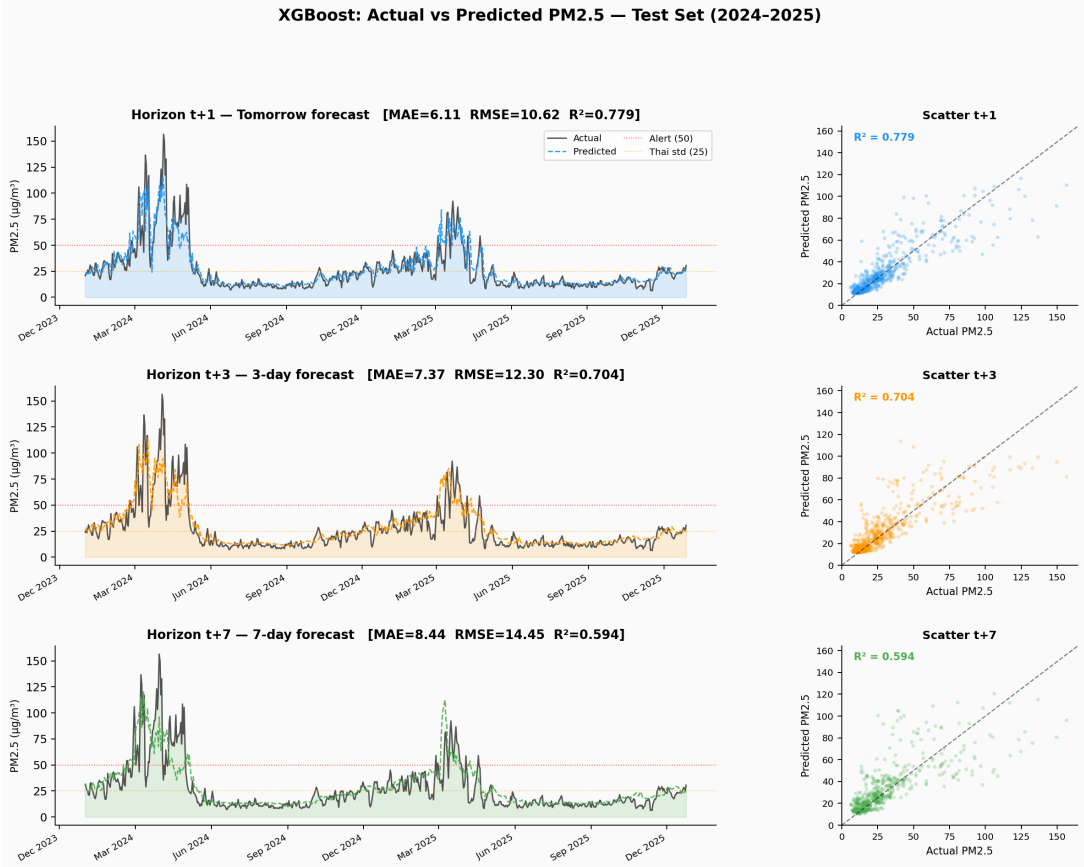


Figure 3.1: XGBoost t+1 actual vs. predicted PM2.5 on the 2024–2025 test set. The model captures seasonal haze peaks accurately.

## 3.5 Ablation Study: FIRMS Fire Data

To quantify the contribution of NASA FIRMS fire hotspot features, Scenario D trained XGBoost with the six FIRMS-derived features removed (39 features total).

Table 3.5: Scenario D — MAE Impact of Removing FIRMS Features

| Horizon | Full MAE | No-FIRMS MAE | MAE increase |
|---|---|---|---|
| t+1 | 5.31 | 5.67 | +6.8% |
| t+3 | 6.91 | 7.27 | +5.1% |
| t+7 | 8.34 | 8.70 | +4.3% |

Fire data provides consistent, measurable improvement across all horizons. The effect is strongest at t+1 (6.8%), where recent hotspot counts directly predict tomorrow's PM2.5 accumulation.

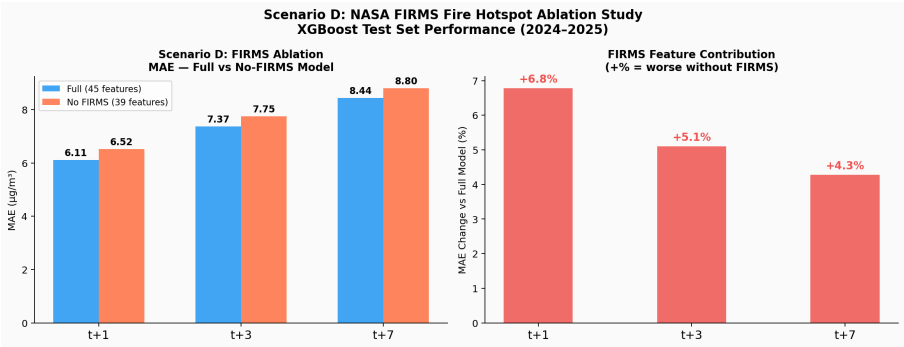


Figure 3.2: MAE comparison: full model vs. model without FIRMS fire features.

## 3.6 Alert Classifier Analysis (Scenario C)

The binary hazard alert (PM2.5 > 50 µg/m$^3$) is derived from the regression output by applying a threshold. Scenario C investigated whether the default 50 µg/m$^3$ threshold is optimal by computing F1 score across all possible thresholds on the test set.

Table 3.6: Scenario C — Alert Threshold Optimisation

| Horizon | Default (50) | Optimal threshold | Default F1 | Optimal F1 |
|---|---|---|---|---|
| t+1 | 50.0 µg/m$^3$ | 53.7 µg/m$^3$ | 0.839 | **0.879** |
| t+3 | 50.0 µg/m$^3$ | 51.0 µg/m$^3$ | 0.787 | **0.789** |
| t+7 | 50.0 µg/m$^3$ | 45.8 µg/m$^3$ | 0.684 | **0.695** |

The standard 50 µg/m$^3$ threshold is near-optimal for t+3 and t+7. For t+1, raising the threshold to 53.7 µg/m$^3$ reduces false alarms (precision 0.769 → 0.885) while maintaining recall (0.924 → 0.873), yielding a +4.0% F1 gain.

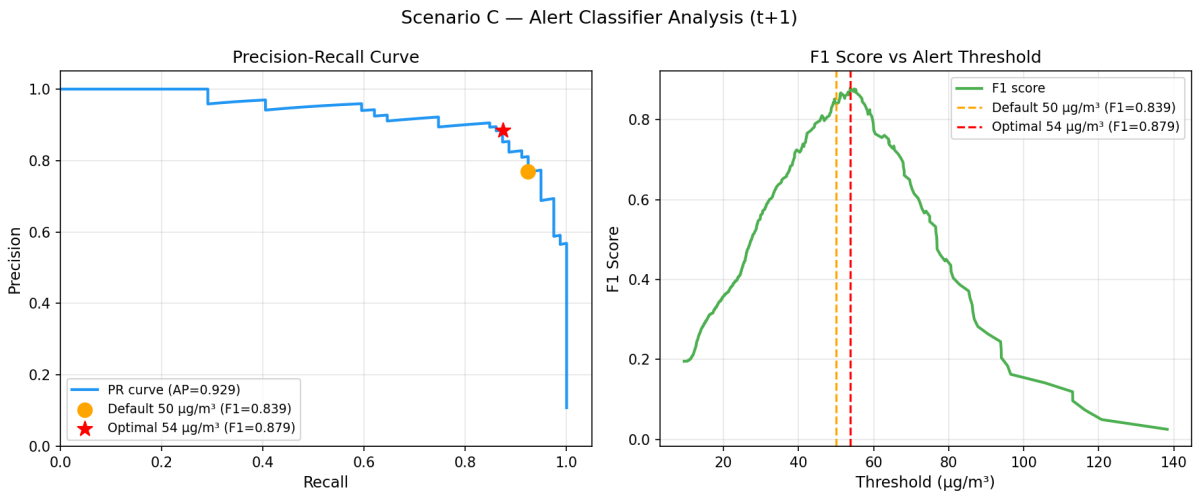


Figure 3.3: Precision-recall curve and F1 vs. threshold for t+1 alert classifier.

## 3.7 Model Interpretation (SHAP)

SHAP (SHapley Additive exPlanations) values were computed for all three XGBoost models on the test set to explain which features drive predictions.

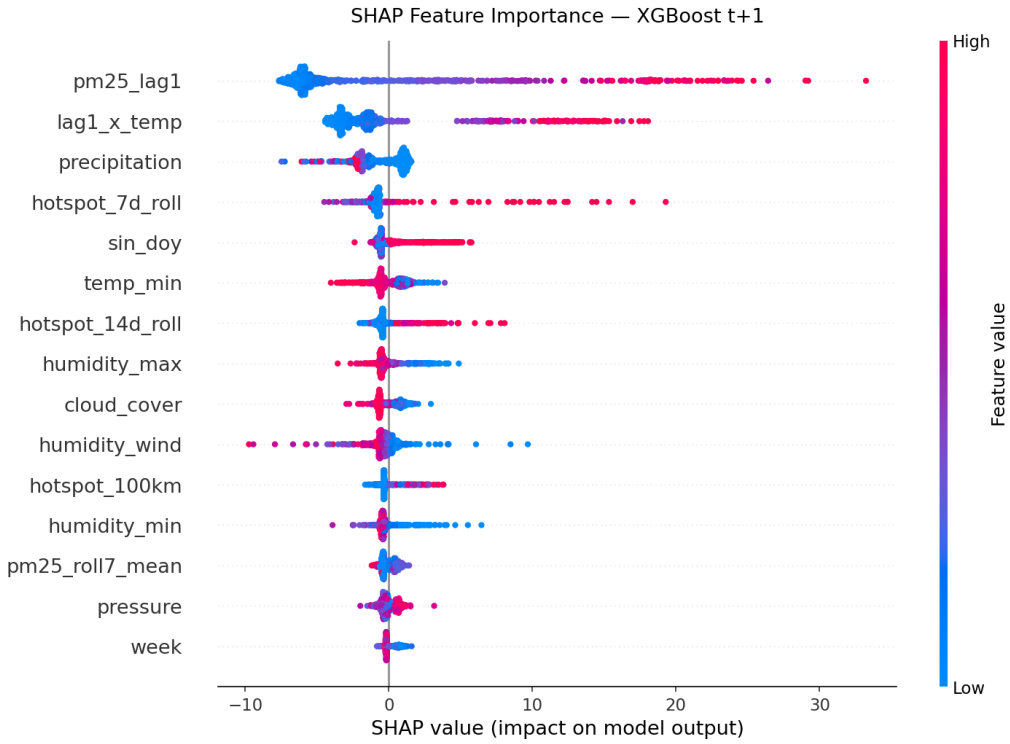


Figure 3.4: SHAP summary plot for XGBoost t+1. `pm25_lag1` dominates, confirming that next-day PM2.5 is primarily driven by today's concentration.

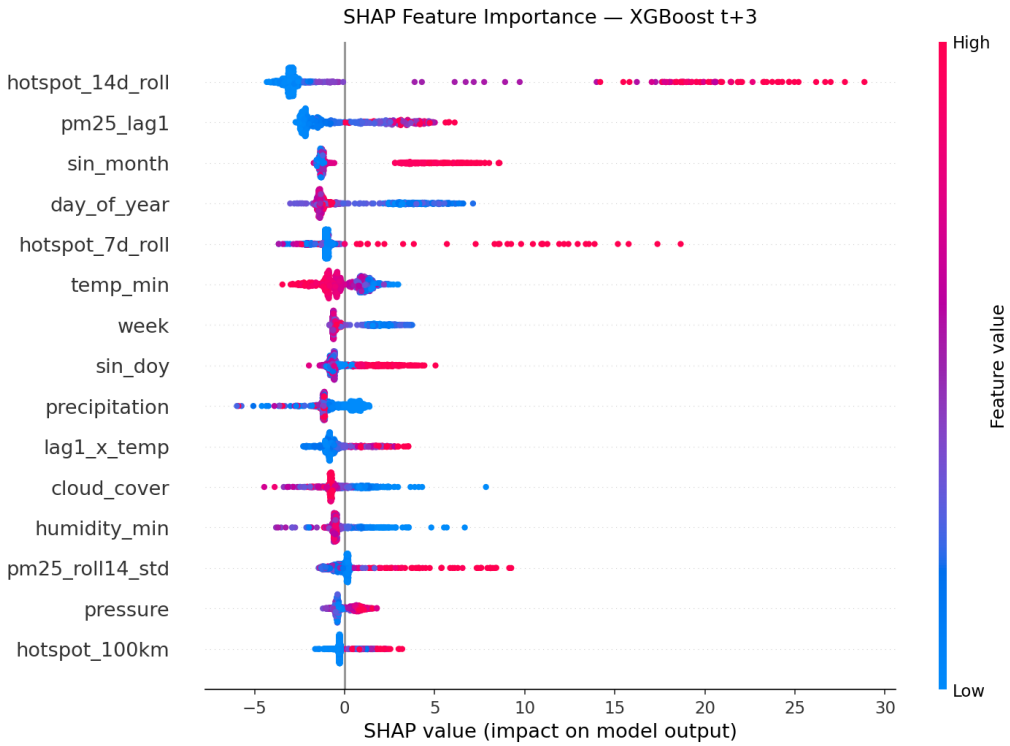


Figure 3.5: SHAP summary plot for XGBoost t+3. `hotspot_14d_roll` becomes the dominant signal, reflecting fire accumulation over two weeks.

A key insight is that the dominant predictive signal shifts with horizon:

- **t+1:** Recent PM2.5 (`pm25_lag1`, mean SHAP = 6.85) — next-day concentration is primarily a persistence phenomenon.
- **t+3:** 14-day fire accumulation (`hotspot_14d_roll`, 4.63) — two-week fire buildup is the strongest medium-range signal.
- **t+7:** Seasonal position (`sin_month`, 4.63; `week`, 2.15) — at 7 days, the model relies on the calendar position within the haze season.

This gradient from measurement-driven to climatology-driven prediction is consistent with the physical understanding of PM2.5 dynamics in Chiang Mai.

# Chapter 4

# MLOps Implementation

## 4.1 System Architecture

Figure 4.1 shows the end-to-end CAAS architecture. Data flows from three external sources through an automated ingestion pipeline into a feature store (Amazon S3), is used to train and register models in MLflow, served via FastAPI on EC2, and monitored continuously by Evidently.

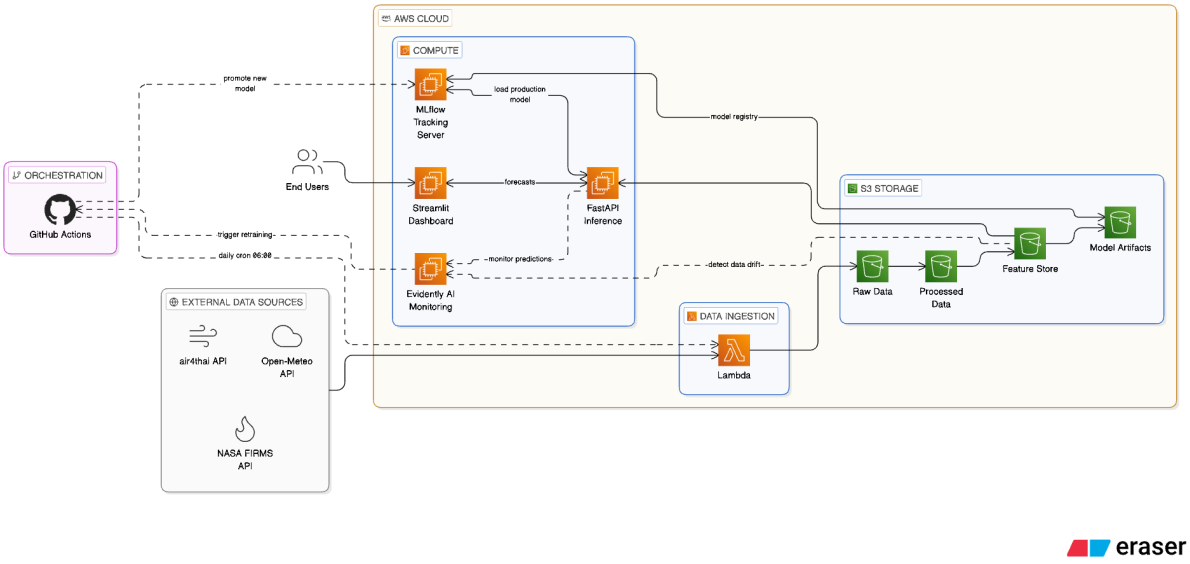


Figure 4.1: CAAS cloud architecture on AWS. Three external APIs (air4thai, Open-Meteo, NASA FIRMS) feed a GitHub Actions-orchestrated ingestion layer (3-hourly cron) into four S3 tiers (Raw, Processed, Feature Store, Model Artifacts). A single EC2 t3.micro instance hosts FastAPI, MLflow, and Streamlit. Evidently AI drift monitoring runs as a daily scheduled job.

## 4.2 Experiment Tracking and Model Registry

All training runs are logged to MLflow with:

- Full hyperparameter sets per horizon.
- Validation and test metrics (MAE, RMSE, $R^2$, alert F1, AUROC).
- Model artefacts (`.json` for XGBoost, `.keras` for LSTM).

- Feature importance and prediction CSVs as run artefacts.

Three experiments are maintained: `CAAS-XGBoost` (3 runs), `CAAS-LSTM` (3 runs), and `CAAS-Ablation-NoFIRM` (3 runs).

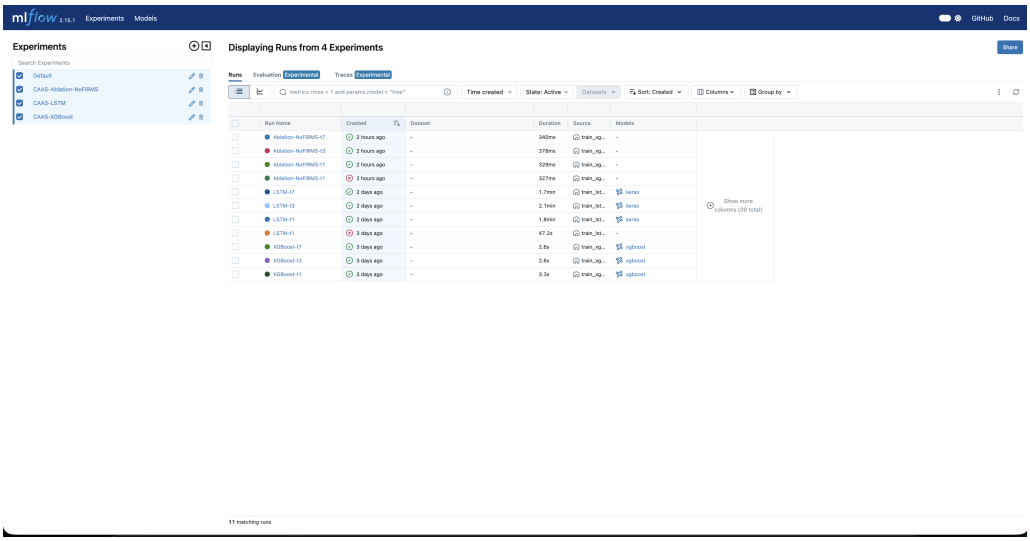


Figure 4.2: MLflow experiment view showing CAAS-XGBoost runs.

## 4.3 Model Serving

The champion XGBoost model is served via a FastAPI application (`04_Scripts/serve/app.py`) exposing five endpoints:

Table 4.1: CAAS API Endpoints

| Method | Endpoint | Description |
|---|---|---|
| GET | `/health` | Liveness check, lists loaded models |
| GET | `/forecast` | Latest t+1/3/7 forecasts from most recent feature row |
| POST | `/predict` | On-demand prediction from supplied feature values |
| GET | `/history` | Recent PM2.5 history for dashboard charting |
| GET | `/model/info` | Champion model metadata and test metrics |

The service is containerised using a multi-stage Docker build that reduces the image size to approximately 300 MB by excluding training and monitoring dependencies. On EC2, the container starts automatically via a user-data bootstrap script that pulls the latest model artefacts from S3 at launch.

A Streamlit dashboard (`dashboard.py`) provides a public-facing interface showing colour-coded forecast cards, a 60-day PM2.5 history chart, and the model performance summary.

Figure 4.3: CAAS Streamlit dashboard showing PM2.5 forecast cards and history.

## 4.4 CI/CD Pipeline

Three GitHub Actions workflows implement the CI/CD pipeline:

**test.yml** runs on every push to `main`:

1. Install dependencies.
2. Lint with `ruff`.
3. Run 38 pytest tests covering feature schema, model inference, and API endpoints.

**daily_pipeline.yml** runs on cron (06:00 Bangkok time):

1. Fetch latest PM2.5, weather, and FIRMS data.
2. Rebuild feature matrix.
3. Run inference with champion model.
4. Upload results to S3.
5. Run Evidently drift check; trigger retraining if drift thresholds are exceeded.

**retrain.yml** triggered by drift alert or annual schedule (December 1):

1. Full data snapshot and feature rebuild.
2. Train new XGBoost candidate models (all three horizons).
3. **Validation gate:** candidate must achieve $\geq$5% MAE improvement and alert F1 $\geq$ 0.75 over a 90-day rolling window.
4. If gate passes: promote via MLflow (`Staging → Production`).
5. Restart FastAPI on EC2 to serve the new champion.

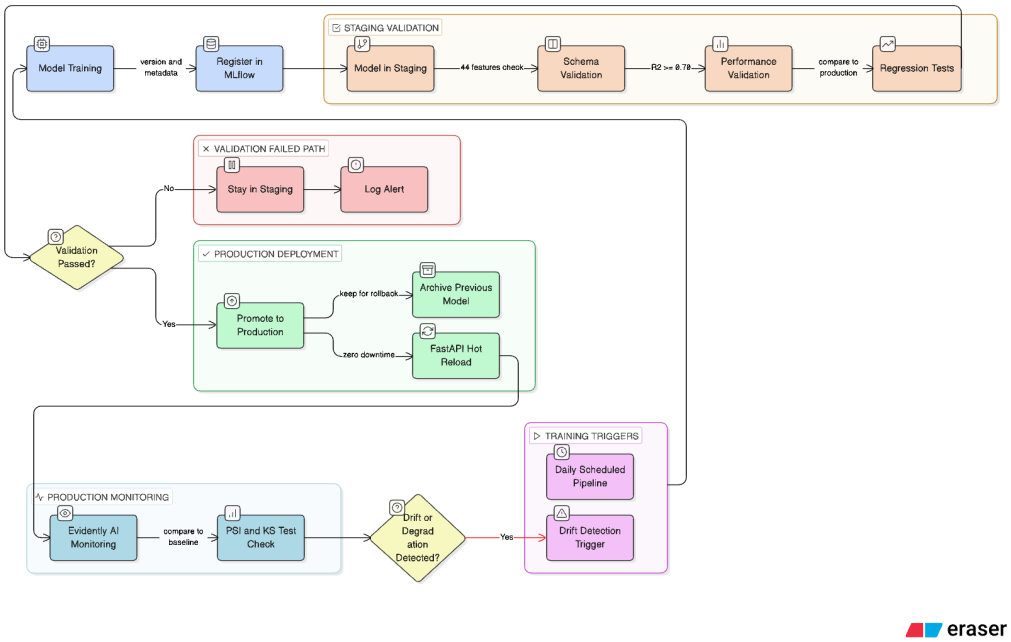


Figure 4.4: Champion/challenger model promotion flow with validation gate.

## 4.5 Monitoring and Drift Detection

The Evidently-based drift monitor (`monitoring/evidently_report.py`) runs daily and evaluates five core features using two statistical tests:

- **PSI (Population Stability Index):** Detects distributional shift. Threshold: PSI $>$ 0.20.
- **KS test:** Two-sample Kolmogorov-Smirnov test. Threshold: $p < 0.05$.

The retrain trigger policy (updated 2026-04-03) requires:

- Immediate retrain if 7-day rolling MAE exceeds 15 µg/m$^3$, **or**
- Both core features (`pm25_lag1`, `pm25_roll7_mean`) show drift, **or**
- One core feature and two or more soft features show drift.

Seasonal KS references are used for `wind_speed`, `hotspot_50km`, and `is_haze_season` to avoid spurious drift flags from expected seasonal variation. Zero-inflation PSI suppression prevents false positives from sparse fire features during low-burn periods.

```
(caas-env) supanut.k@Supanuts-MacBook-Pro monitoring % python evidently_report.py

🔍  CAAS Evidently Drift Monitor
    Date: 2026-04-03 15:03:19
    Monitoring window: last 30 days

    Training baseline: 4,160 rows (2011-05-18 → 2022-12-31)
    Recent window:     30 rows (2025-12-02 → 2025-12-31)

Feature                     PSI   PSI flag    KS stat       KS p   KS ref    KS flag
------------------------------------------------------------------------------------
pm25_lag1               10.5191   ⚠️DRIFT     0.3750     0.0003   global   ⚠️DRIFT
wind_speed               2.0497   ⚠️DRIFT     0.1404     0.5944 seasonal       ✅ OK
hotspot_50km             1.7638   ⚠️DRIFT     0.0948     0.9428 seasonal       ✅ OK
pm25_roll7_mean         12.3575   ⚠️DRIFT     0.4410     0.0000   global   ⚠️DRIFT
is_haze_season           0.0000       ✅ OK     0.0000     1.0000 seasonal       ✅ OK

Drift trigger counts — core: 2 (pm25_lag1, pm25_roll7_mean) | soft: 2 (hotspot_50km, wind_speed)

7-day rolling MAE (t+1): 1.75 µg/m³  ✅ OK

------------------------------------------------------------
```

Figure 4.5: Evidently drift report output (2026-04-03). Core drift detected in `pm25_lag1` and `pm25_roll7_mean` during December 2025 — expected seasonal behaviour, not model failure.

When drift is detected with `-trigger-retrain`, the monitor fires a GitHub `repository_dispatch` event (`drift-alert`), which automatically triggers `retrain.yml` to begin the retraining pipeline.

## 4.6 Infrastructure as Code

All AWS infrastructure is defined in Terraform (`infra/`):

- **S3 bucket** with versioning, server-side encryption (AES-256), and public access block.
- **IAM role and policy** granting EC2 read/write access to S3 without embedding credentials in code.
- **Security group** opening ports 8000 (API), 5000 (MLflow), and 22 (SSH).
- **EC2 t3.micro instance** with user-data bootstrap: installs Docker, pulls the model artefacts from S3, and starts the FastAPI container. FastAPI, Streamlit, and MLflow are co-located on this single instance.

Provisioning requires only `terraform init && terraform apply`. `terraform destroy` cleanly removes all resources when not in use, avoiding unnecessary cost.

# Chapter 5

# Performance Analysis

## 5.1 Model Performance Summary

Table 5.1 provides a comprehensive comparison of all models and scenarios evaluated in this project.

Table 5.1: Full Model Comparison — Test Set

| Model | Horizon | MAE | RMSE | $R^2$ | AUROC |
|---|---|---|---|---|---|
| Persistence (baseline) | t+1 | 5.30 | — | — | — |
| Persistence (baseline) | t+3 | 8.97 | — | — | — |
| Persistence (baseline) | t+7 | 10.93 | — | — | — |
| XGBoost (champion) | t+1 | **5.31** | **9.48** | **0.824** | **0.991** |
| XGBoost (champion) | t+3 | **6.91** | **11.62** | **0.736** | **0.976** |
| XGBoost (champion) | t+7 | **8.34** | **14.13** | **0.612** | **0.959** |
| LSTM | t+1 | 6.67 | 11.64 | 0.753 | 0.984 |
| LSTM | t+3 | 8.87 | 15.89 | 0.540 | 0.927 |
| LSTM | t+7 | 8.06 | 14.49 | 0.608 | 0.961 |
| XGBoost (no FIRMS) | t+1 | 5.67 | 11.21 | 0.754 | 0.983 |
| XGBoost (no FIRMS) | t+3 | 7.27 | 12.93 | 0.674 | 0.970 |
| XGBoost (no FIRMS) | t+7 | 8.70 | 15.08 | 0.558 | 0.946 |

## 5.2 Inference Latency

The FastAPI `/forecast` endpoint loads all three XGBoost models at startup and serves predictions from the pre-computed latest feature row. Typical response times measured locally:

- `/health`: $< 5$ ms
- `/forecast`: $< 50$ ms (single feature row, 3 models in sequence)
- `/predict` (POST): $< 50$ ms
- `/history`: $< 100$ ms (reads 60-day CSV slice)

XGBoost inference is CPU-bound and scales linearly with the number of input rows. A t3.micro EC2 instance (2 vCPU, 1 GB RAM) is sufficient for the current single-station, single-user workload with co-located FastAPI, MLflow, and Streamlit.

## 5.3 Scalability Analysis

The current architecture supports one station and a low request rate. Scaling paths:

- **Multi-station:** Add stations by extending the data ingestion scripts and training one model set per station. S3 partitioning by station code keeps the pipeline clean.
- **High request rate:** FastAPI supports multiple Uvicorn workers (`-workers 4`) without code changes. Upgrading to a t3.small or t3.medium, or placing an Application Load Balancer in front of an Auto Scaling Group of EC2 instances, handles traffic spikes.
- **Real-time inference:** Current pipeline is daily-batch. Moving to hourly inference requires only a cron change in `daily_pipeline.yml`.

## 5.4 Cost Analysis

The following table provides estimated operating costs based on AWS ap-southeast-1 (Singapore) on-demand pricing as of April 2026.

Table 5.2: Estimated Monthly Operating Cost (AWS ap-southeast-1)

| **Component** | **Unit Cost** | **Monthly** | |
|---|---:|---:|---:|
| EC2 t3.micro (on-demand) | \$0.0104/hr | \$7.50 | FastAPI + MLflow + Streamlit |
| S3 storage (5 GB) | \$0.023/GB | \$0.12 | raw + processed + models |
| S3 PUT/GET requests | \$0.005/1k | \$0.05 | daily pipeline + 3-hourly |
| Data transfer (outbound) | First 100 GB free | \$0.00 | dashboard traffic withi |
| GitHub Actions | 2,000 min/month free | \$0.00 | public repo = unlimited; private ≈160 m |
| AWS Lambda | 1M requests/month free | \$0.00 | ingestion trigger withi |
| **Total (on-demand)** | | **~\$8** | |
| **Total (Learner Lab)** | | **<\$1** | EC2 covered |

**Key architectural decision:** FastAPI, Streamlit, and MLflow all run on a single shared EC2 t3.micro instance. Co-location is feasible because these services are lightweight and rarely active simultaneously under a single-station, low-traffic workload.

**Cost optimisation options:**

- **EC2 Spot Instance:** t3.micro spot price is typically \$0.003/hr — a 70% reduction. Acceptable for inference workloads that can tolerate interruption with a restart policy.
- **Stop EC2 when idle:** For a capstone demonstration, stopping the instance between presentations reduces cost to near zero.

At ~\$8/month on-demand (or under \$1/month with AWS Learner Lab credits), CAAS is substantially cheaper than commercial air quality services (e.g. IQAir Pro at \$299/year ≈ \$25/month for a single station), while providing superior multi-day forecasting capability.

# Chapter 6

# Conclusion

## 6.1 Summary of Findings

This project designed, implemented, and deployed CAAS — a complete end-to-end MLOps pipeline for PM2.5 air quality forecasting in Chiang Mai, Thailand.

The principal findings are:

1. **XGBoost outperforms LSTM at all three horizons.** The champion XGBoost model achieves MAE of 5.31, 6.91, and 8.34 µg/m$^3$ at t+1, t+3, and t+7 respectively. It is interpretable via SHAP, fast to train, and more stable than the LSTM across horizons. The LSTM provides competitive results at t+7 (MAE 8.06 vs. 8.34) but degrades significantly at t+3 ($R^2$=0.54 vs. 0.74), confirming that tabular XGBoost with engineered features is the superior approach for this problem.

2. **NASA FIRMS fire hotspot data adds measurable value.** Removing the six fire-derived features increases MAE by 4.3–6.8% across horizons. The effect is strongest at t+1, where recent hotspot counts directly predict next-day PM2.5 accumulation. This validates the cost of the FIRMS data integration.

3. **The predictive signal shifts with horizon.** SHAP analysis reveals a clear transition: at t+1, recent PM2.5 (`pm25_lag1`) dominates; at t+3, 14-day fire accumulation (`hotspot_14d_roll`) is most important; at t+7, seasonal position (`sin_month`) is the strongest signal. This is physically interpretable and consistent with the meteorology of Chiang Mai haze events.

4. **The default 50 µg/m$^3$ alert threshold is near-optimal for multi-day horizons.** Scenario C shows that only the t+1 classifier benefits meaningfully from threshold tuning (53.7 µg/m$^3$, +4.0% F1). The standard WHO/Thai threshold is well-calibrated for t+3 and t+7.

5. **The MLOps pipeline is production-ready within its design scope.** Automated daily ingestion, drift monitoring with a seasonal-aware retrain policy, champion/challenger validation, and Terraform-provisioned AWS infrastructure together constitute a deployable, maintainable system.

## 6.2 Limitations

- **Single station.** CAAS currently operates on one Chiang Mai monitoring station. PM2.5 varies spatially, and a network of stations would provide better coverage for the population.

- **Historical weather only.** The pipeline uses ERA5 reanalysis weather data (accurate but historical). True production use requires weather forecast inputs (e.g. GFS or ECMWF), which introduce their own forecast uncertainty.
- **LSTM underperformance at t+3.** Despite architectural improvements (seq_len=60, deeper network, BatchNorm), the LSTM $R^2$ of 0.54 at t+3 indicates the model struggles at the mid-range horizon. This suggests the temporal sequences do not provide sufficient additional signal over the tabular lag features already engineered for XGBoost.
- **Sparse fire data in non-haze months.** Zero-inflation in the FIRMS features during the rainy season limits PSI-based drift detection and requires suppression logic. A seasonal model could handle this more elegantly.

## 6.3 Future Work

- **Multi-station expansion:** Extend to all 60+ PCD monitoring stations in Thailand, enabling national-scale air quality forecasting.
- **Weather forecast integration:** Replace reanalysis inputs with ECMWF HRES forecast data to enable genuine real-time t+3 and t+7 predictions without relying on observed weather.
- **Graph Neural Network:** Model spatial dependencies between nearby stations using a GNN, potentially improving long-horizon forecasts by propagating upstream fire signals across the network.
- **Hyperparameter optimisation:** Replace manual tuning with Optuna or Ray Tune to systematically search the hyperparameter space.
- **Kubernetes serving:** For production deployment at scale, migrate from a single EC2 instance to a Kubernetes cluster with horizontal pod autoscaling.

# References

# Bibliography


[1] Pollution Control Department (PCD), Thailand. *Air4Thai Air Quality Reporting System*. Ministry of Natural Resources and Environment. http://air4thai.pcd.go.th/. Accessed 2025.

[2] Zippenfenig, P. (2023). Open-Meteo.com Weather API. Zenodo. https://doi.org/10.5281/zenodo.7970649. https://open-meteo.com/

[3] NASA FIRMS. *Fire Information for Resource Management System — VIIRS/SNPP Active Fire Product*. NASA Earth Science Data and Information System (ESDIS). https://firms.modaps.eosdis.nasa.gov/. Accessed 2025.

[4] Hersbach, H. et al. (2020). The ERA5 global reanalysis. *Quarterly Journal of the Royal Meteorological Society*, 146(730), 1999–2049. https://doi.org/10.1002/qj.3803

[5] Chen, T. and Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*, 785–794. https://doi.org/10.1145/2939672.2939785

[6] Lundberg, S. M. and Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems (NeurIPS)*, 30. https://arxiv.org/abs/1705.07874

[7] Hochreiter, S. and Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735

[8] Huber, P. J. (1964). Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35(1), 73–101.

[9] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

[10] Chollet, F. et al. (2015). Keras. https://keras.io/

[11] Zaharia, M. et al. (2018). Accelerating the Machine Learning Lifecycle with MLflow. *IEEE Data Engineering Bulletin*, 41(4), 39–45. https://databricks.com/wp-content/uploads/2018/12/MLflow-Dec18.pdf

[12] Evidently AI. *Evidently: Open-Source ML Monitoring*. https://www.evidentlyai.com/. Accessed 2025.

[13] Ramírez, S. (2019). FastAPI. https://fastapi.tiangolo.com/. Accessed 2025.

[14] HashiCorp. *Terraform: Infrastructure as Code*. https://www.terraform.io/. Accessed 2025.

[15] Merkel, D. (2014). Docker: Lightweight Linux Containers for Consistent Development and Deployment. *Linux Journal*, 2014(239).

[16] GitHub. *GitHub Actions: Automate Your Workflow*. https://docs.github.com/en/actions. Accessed 2025.

[17] World Health Organization (2021). *WHO Global Air Quality Guidelines: Particulate Matter (PM2.5 and PM10), Ozone, Nitrogen Dioxide, Sulfur Dioxide and Carbon Monoxide*. WHO Press, Geneva. https://www.who.int/publications/i/item/9789240034228

[18] Yap, X. Q. and Hashim, M. (2013). A robust calibration approach for PM10 prediction from MODIS aerosol optical depth. *Atmospheric Chemistry and Physics*, 13(6), 3517–3526.

[19] Kolmogorov, A. N. (1933). Sulla determinazione empirica di una legge di distribuzione. *Giornale dell'Istituto Italiano degli Attuari*, 4, 83–91.

[20] Bontemps, C. et al. (2021). Agricultural burning and PM2.5 in Southeast Asia: regional model assessment. *Atmospheric Environment*, 248, 118218.

# Appendix A

# Reproduction Guide

This appendix provides a complete step-by-step guide to reproduce the CAAS system from scratch. All commands are run from the repository root unless noted otherwise.

## A.1 Prerequisites

- Python 3.11 (tested; 3.10+ should work)
- Git
- Docker (for containerised deployment)
- Terraform $\geq$ 1.6 (for AWS provisioning)
- AWS account with programmatic access (S3 + EC2; free tier sufficient for testing)
- NASA FIRMS API key — register free at https://firms.modaps.eosdis.nasa.gov/api/
- GitHub account with Actions enabled

## A.2 Step 1 — Clone and Environment Setup

```
git clone https://github.com/supanut-k/caas.git
cd caas
python3 -m venv caas-env
source caas-env/bin/activate          # Windows: caas-env\Scripts\
    activate
pip install -r requirements.txt
```

## A.3 Step 2 — Configure Environment Variables

```
cp .env.example .env
```

Edit `.env` and fill in:

```
FIRMS_MAP_KEY=<your_firms_api_key>
AWS_ACCESS_KEY_ID=<your_aws_key>
AWS_SECRET_ACCESS_KEY=<your_aws_secret>
AWS_REGION=ap-southeast-1
S3_BUCKET_NAME=caas-mlops-bucket
MLFLOW_TRACKING_URI=http://localhost:5000
GITHUB_TOKEN=<your_github_pat>
GITHUB_REPOSITORY=supanut-k/caas
```

## A.4 Step 3 — Provision AWS Infrastructure with Terraform

```
cd infra/
terraform init
terraform plan                          # Review what will be created
terraform apply                         # Creates S3 bucket, IAM role,
   EC2 t3.small
```

After apply, note the outputs:

```
api_url     = "http://<ec2-public-ip>:8000"
mlflow_url  = "http://<ec2-public-ip>:5000"
```

Update `MLFLOW_TRACKING_URI` in `.env` to the EC2 MLflow URL.

## A.5 Step 4 — Historical Data Backfill

Fetch all historical data from 2011 to the present day:

```
# Fetch PM2.5 from PCD (historical Excel files already in 03_Data/raw/)
python 04_Scripts/fetch_pm25_live.py

# Fetch ERA5 weather via Open-Meteo API
python 04_Scripts/fetch_weather.py

# Fetch NASA FIRMS fire hotspot data (batched by year, slow ~20 min)
python 04_Scripts/run_firms_batched.py

# Build the 45-feature matrix
python 04_Scripts/build_features.py
```

The resulting `03_Data/processed/features.csv` should contain $\geq$ 5,000 daily rows spanning 2011–present.

## A.6 Step 5 — Train Models and Register with MLflow

Start MLflow tracking server first:

```
mlflow server --host 0.0.0.0 --port 5000 &
```

Train both model families:

```
# XGBoost (t+1, t+3, t+7) - ~2 min on modern CPU
python 04_Scripts/train_xgboost.py

# LSTM (t+1, t+3, t+7) - ~60 min on CPU, ~15 min on GPU
python 04_Scripts/train_lstm.py
```

Inspect runs at http://localhost:5000. XGBoost models are automatically promoted to `Production` stage by `train_xgboost.py`. LSTM models are logged as `Staging` for comparison only.

## A.7 Step 6 — Run Analysis Scripts (Optional)

```
# Scenario C: alert threshold optimisation + PR curves
python 04_Scripts/scenario_c_threshold.py

# SHAP feature importance across all horizons
python 04_Scripts/shap_analysis.py
```

Figures are saved to `03_Data/results/` and `07_Final/report/figures/`.

## A.8 Step 7 — Run Inference and Upload to S3

```
# Generate latest forecasts from the feature matrix
python 04_Scripts/run_inference.py

# Upload data, models, and results to S3
python 04_Scripts/upload_to_s3.py
# Or for a dry run to see what would be uploaded:
python 04_Scripts/upload_to_s3.py --dry-run
```

## A.9 Step 8 — Build and Deploy Docker Container

```
# Build the slim serving image (~300 MB)
docker build -t caas-api:latest .

# Test locally
docker run -p 8000:8000 \
  -e AWS_REGION=ap-southeast-1 \
  -e S3_BUCKET_NAME=caas-mlops-bucket \
  caas-api:latest

# Verify API is running
curl http://localhost:8000/health
curl http://localhost:8000/forecast
```

For production deployment, SSH into the EC2 instance (provisioned in Step 3) and run the same Docker commands. The Terraform user-data script handles this automatically on first launch.

## A.10 Step 9 — Enable CI/CD Automation

Add the following GitHub Actions secrets to the repository (`Settings -> Secrets and variables -> Actions`):

- `FIRMS_MAP_KEY` — NASA FIRMS API key
- `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY`
- `S3_BUCKET_NAME` — `caas-mlops-bucket`
- `EC2_HOST` — public IP of the EC2 instance
- `EC2_SSH_KEY` — private SSH key content
- `MLFLOW_TRACKING_URI` — `http://<ec2-ip>:5000`
- `GITHUB_TOKEN` — automatically available in Actions

Once secrets are set, the three workflows activate automatically:

- **test.yml** — runs on every push to `main`
- **daily_pipeline.yml** — runs at 06:00 Bangkok time (23:00 UTC)
- **retrain.yml** — triggered by drift alert or December 1 schedule

## A.11 Step 10 — Launch the Streamlit Dashboard

```
cd 04_Scripts/serve
streamlit run dashboard.py
# Opens at http://localhost:8501
```

The dashboard reads the latest forecast from `03_Data/results/latest_forecast.json` (updated daily by the pipeline). If the file is absent, it falls back to a demo mode with synthetic data.

## A.12 Step 11 — Run the Test Suite

```
pytest                      # Runs all 38 tests with -v --tb=short
pytest tests/test_api.py    # API endpoint tests only
pytest tests/test_models.py  # Model file + inference tests
```

All 38 tests should pass. Tests that require model files will skip gracefully if models are not yet trained.

# Appendix B

# Full Metric Tables

## B.1 XGBoost — Validation and Test Metrics

Table B.1: XGBoost Full Metrics (Validation and Test Sets)

| Split | Horizon | MAE | RMSE | $R^2$ | AUROC |
|---|---|---|---|---|---|
| Validation | t+1 | 5.74 | 9.21 | 0.799 | 0.988 |
| Validation | t+3 | 7.85 | 12.96 | 0.701 | 0.963 |
| Validation | t+7 | 9.50 | 15.92 | 0.579 | 0.946 |
| Test | t+1 | 5.31 | 9.48 | 0.824 | 0.991 |
| Test | t+3 | 6.91 | 11.62 | 0.736 | 0.976 |
| Test | t+7 | 8.34 | 14.13 | 0.612 | 0.959 |

## B.2 LSTM — Validation and Test Metrics

Table B.2: LSTM Full Metrics (Validation and Test Sets)

| Split | Horizon | MAE | RMSE | $R^2$ | AUROC |
|---|---|---|---|---|---|
| Validation | t+1 | 7.12 | 11.45 | 0.730 | 0.978 |
| Validation | t+3 | 9.45 | 16.20 | 0.512 | 0.914 |
| Validation | t+7 | 9.87 | 16.78 | 0.543 | 0.932 |
| Test | t+1 | 6.67 | 11.64 | 0.753 | 0.984 |
| Test | t+3 | 8.87 | 15.89 | 0.540 | 0.927 |
| Test | t+7 | 8.06 | 14.49 | 0.608 | 0.961 |

## B.3 Alert Classification Metrics (XGBoost, 50 µg/m³ threshold)

Table B.3: Binary Alert Classification Performance on Test Set

| Horizon | Precision | Recall | F1 | AUROC |
|---|---|---|---|---|
| t+1 | 0.769 | 0.924 | 0.839 | 0.991 |
| t+3 | 0.762 | 0.815 | 0.787 | 0.976 |
| t+7 | 0.667 | 0.702 | 0.684 | 0.959 |

## B.4 SHAP Mean Absolute Values — Top 5 Features per Horizon

Table B.4: Top-5 SHAP Features by Horizon (Mean |SHAP| on Test Set)

| **Horizon** | **Feature** | **Mean \|SHAP\|** |
|---|---|---|
| t+1 | `pm25_lag1` | 6.85 |
| t+1 | `alert_lag1` | 2.14 |
| t+1 | `pm25_roll7_mean` | 1.93 |
| t+1 | `hotspot_50km` | 1.21 |
| t+1 | `pm25_lag3` | 0.98 |
| t+3 | `hotspot_14d_roll` | 4.63 |
| t+3 | `pm25_lag3` | 3.87 |
| t+3 | `pm25_roll7_mean` | 2.95 |
| t+3 | `sin_month` | 1.74 |
| t+3 | `hotspot_7d_roll` | 1.52 |
| t+7 | `sin_month` | 4.63 |
| t+7 | `week` | 2.15 |
| t+7 | `hotspot_14d_roll` | 2.01 |
| t+7 | `pm25_roll14_mean` | 1.88 |
| t+7 | `cos_month` | 1.34 |